

SRINIVAS UNIVERSITY
INSTITUTE OF ENGINEERING & TECHNOLOGY
MUKKA, MANGALURU-574146



MINI PROJECT REPORT

ON

Campus Issue Raiser

Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE & ENGINEERING

Submitted By,

Jyothsna R Jogi
Lishmith

01SU23CS083
01SU23CS092

Under the guidance of,

Mr. Yogesh Bandewar

IBM Trainer

Dept. of CSE

2025-2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

SRINIVAS UNIVERSITY, MUKKA

SRINIVAS UNIVERSITY
INSTITUTE OF ENGINEERING & TECHNOLOGY
MUKKA, MANGALURU-574146



Department of Cyber Security & Cyber Forensics

CERTIFICATE

This is to certify that the Technical Seminar entitled ***“Campus Issue Raiser”*** is a bonafide work carried out by ***Jyothsna R Jogi ,Lishmith*** bearing the USN ***01SU23CS083, 01SU23CS092*** in the partial fulfillment for the award of **Bachelor of Technology in Computer Science & Engineering** of the Srinivas University- Institute of Engineering & Technology during the year **2025-26**. It is certified that all corrections/suggestions indicated for internal assessment have been incorporated in the report deposited in the department library. The mini project report has been approved as it meets the academic criteria outlined for the said degree.

Name & Signature

Name & Signature

Mr. Yogesh Bandewar
(Project Coordinator)

Dr.Shifana Begaum
(HOD, CSE)

SRINIVAS UNIVERSITY
INSTITUTE OF ENGINEERING & TECHNOLOGY
MUKKA, MANGALURU-574146



Department of Cyber Security & Cyber Forensics

DECLARATION

We ***Jyothsna R Jogi ,Lishmith*** the student of 6th semester, **Bachelor of Technology** in **Computer Science & Engineering** of the **Srinivas University- Institute of Engineering & Technology**, Mukka, hereby declare that the mini project report entitled ***“Campus Issue Raiser”*** has been successfully completed by us in partial fulfillment of the requirements for the award of degree in **Bachelor of Technology** in **Computer Science & Engineering** of **Srinivas University- Institute of Engineering & Technology** and no part of it has been submitted for the award of degree or diploma in any university or institution previously.

Date: 28-03-2026

Place: Mukka

ABSTRACT

Campus Issue Raiser is a web-based issue management system designed to streamline the process of reporting, tracking, and resolving campus-related problems. The platform enables students to raise issues, attach evidence, and interact with other users, while authorities and administrators can monitor, manage, and resolve these issues efficiently.

The system is built using a **microservices architecture** with an API Gateway acting as a centralized entry point. Each service handles a specific functionality such as user management, issue tracking, interactions, escalation, and analytics. This modular design improves scalability, maintainability, and flexibility.

The platform also supports role-based access control, issue escalation mechanisms, and analytics dashboards for monitoring performance. By ensuring transparency and accountability, the system enhances communication between students and authorities and provides an efficient solution for institutional issue management.

.

ACKNOWLEDGEMENT

It is our pleasure to acknowledge for all those who have provided guidance, inspiration and encouragement for the successful completion of our mini project.

First and foremost, Our sincere thanks go to **Mr. Yogesh Bandewar** , Mini Project Coordinator, for coordinating the project process, offering timely suggestions, and being a constant source of motivation and assistance.

We are immensely grateful to **Dr.Shifana Begaum** Head of the Department of CSE , for her constant encouragement and valuable support throughout the course of our studies.

We would also like to extend our heartfelt thanks to **Dr. Ramakrishna N Hegde**, Dean, SUIET for his guidance and for facilitating the smooth conduct of the Mini Project Demonstration.

We are deeply thankful to the Management of **Srinivas University Institute of Engineering and Technology** for providing us with the opportunity to undertake this project as part of the academic curriculum. Their vision and commitment to practical learning have made this experience possible.

We would also like to acknowledge the support and encouragement of all the faculty members of the Department of CSE, whose teachings laid the foundation for this practical learning.

Last but not least, We thank our family and friends for their continuous moral support and encouragement during the entire period of the project and report preparation.

This Mini Project has been a truly enriching experience and a vital step in my professional development.

CONTENTS

Chapter 1	INTRODUCTION	7
Chapter 2	PROBLEM STATEMENT	8
Chapter 3	OBJECTIVES OF THE PROJECT	9
Chapter 4	SYSTEM ARCHITECTURE	10-11
Chapter 5	TECHNOLOGIES USED	12-13
Chapter 6	METHODOLOGY	14-15
Chapter 7	IMPLEMENTATION	16-17
Chapter 8	RESULT	18-19
Chapter 9	ADVANTAGES & LIMITATIONS	20
Chapter 10	FUTURE ENHANCEMENTS	21
Chapter 11	CONCLUSION	22
Chapter 12	REFERENCES	23

Chapter-1

INTRODUCTION

In today's educational institutions, students often face various issues related to infrastructure, academics, facilities, and administration. Traditional methods of reporting these problems, such as complaint registers or verbal communication, are inefficient and lack transparency. These methods do not provide proper tracking, resulting in delays and unresolved issues.

To overcome these challenges, the **Campus Issue Raiser** system is developed as a web-based platform that allows students to raise issues easily and track their progress. The system provides a structured way for users to submit complaints, attach necessary details, and monitor the status of their issues.

The platform also enables administrators and authorities to manage and resolve issues efficiently. It includes features such as role-based access, issue status updates, and escalation mechanisms for unresolved complaints. This ensures accountability and improves communication between students and management.

The system is designed using modern web technologies and follows a microservices architecture, which improves scalability, flexibility, and maintainability. Overall, the Campus Issue Raiser system aims to provide a transparent, efficient, and user-friendly solution for managing campus-related issues.

Chapter 2

PROBLEM STATEMENT

In many educational institutions, there is no efficient and structured system for reporting and managing campus-related issues. Students usually depend on traditional methods such as complaint registers, emails, or verbal communication to report problems. These approaches are time-consuming, unorganized, and lack proper documentation.

A major drawback of existing systems is the absence of a real-time tracking mechanism. Once a complaint is submitted, students are unable to track its progress or receive updates regarding its status. This lack of visibility creates confusion, reduces trust in the system, and often leads to repeated complaints for the same issue.

Furthermore, there is no proper categorization or prioritization of issues. Critical problems such as safety concerns or infrastructure failures may not receive immediate attention due to the absence of a structured workflow. The communication gap between students and authorities also results in delays and inefficiencies in resolving issues.

Another significant limitation is the lack of an escalation mechanism. Issues that remain unresolved for a long period are not automatically forwarded to higher authorities, leading to negligence and poor accountability. Additionally, there is no centralized platform to analyze data, monitor performance, or identify recurring problems within the campus.

Hence, there is a strong need for a centralized, transparent, and efficient system that allows users to report issues, track their status in real time, enable interaction, ensure timely escalation, and improve overall accountability. The proposed Campus Issue Raiser system aims to address these challenges by providing a modern, scalable, and user-friendly solution.

OBJECTIVES OF THE PROJECT

1. To develop a centralized campus issue management system

Design and implement a web-based platform that enables students to report and manage campus-related issues in a structured manner.

2. To provide real-time issue tracking

Allow users to track the status of their complaints from submission to resolution, ensuring transparency and visibility.

3. To implement role-based access control

Define different user roles such as students, administrators, and authorities to ensure secure and organized system access.

4. To enable user interaction and engagement

Provide features like comments and support (upvotes) so that users can interact and highlight important issues.

5. To implement an issue escalation mechanism

Automatically escalate unresolved issues to higher authorities after a specific time to ensure timely action.

6. To improve communication between students and authorities

Create a direct and efficient communication channel to reduce delays and misunderstandings.

7. To provide efficient issue management for administrators

Enable authorities to assign, monitor, and resolve issues effectively through a structured workflow.

8. To ensure transparency and accountability

Maintain proper tracking and visibility of issues to hold responsible authorities accountable.

9. To build a scalable and flexible system

Use microservices architecture to ensure the system can handle growth and future enhancements.

SYSTEM ARCHITECTURE

The Campus Issue Raiser system is designed using a **microservices architecture**, which divides the application into multiple independent services. Each service is responsible for a specific functionality and communicates with other services through APIs. This approach improves scalability, flexibility, and maintainability of the system.

◆ Overall Architecture

The system consists of three main layers:

1. Frontend Layer

The frontend is developed using React.js and provides a user-friendly interface for students and administrators. Users can register, log in, raise issues, view issue status, and interact through comments and support features.

2. API Gateway

The API Gateway acts as a single entry point for all client requests. It routes incoming requests to the appropriate microservices and handles tasks such as request validation and authentication. This ensures smooth communication between the frontend and backend services.

3. Backend Microservices Layer

The backend is divided into multiple independent services, each handling a specific function:

- **User Service** – Manages user registration, login, and authentication
- **Issue Service** – Handles issue creation, updates, and tracking
- **Interaction Service** – Manages comments and support (upvotes)
- **Escalation Service** – Handles automatic escalation of unresolved issues
- **Status Service** – Tracks and updates issue status
- **Evidence Service** – Manages file uploads such as images or documents
- **Analytics Service** – Provides reports and insights on issues

Each microservice communicates via REST APIs and operates independently, allowing easier updates and scalability.

4. Database Layer

The system uses a database (such as MongoDB or JSON-based storage) to store user data, issues, interactions, and logs. Although a shared database is used in the current implementation, it can be extended to separate databases for each microservice in the future.

◆ Key Features of the Architecture

- **Scalability** – Services can be scaled independently
- **Flexibility** – Easy to add or modify features
- **Fault Isolation** – Failure in one service does not affect others
- **Maintainability** – Code is modular and easier to manage

◆ Working Flow

1. User sends request from frontend
2. Request goes to API Gateway
3. Gateway routes request to the appropriate microservice
4. Microservice processes request and interacts with database
5. Response is sent back to the user through the gateway

Chapter - 5

TECHNOLOGIES USED

The Campus Issue Raiser system is developed using modern web technologies to ensure efficiency, scalability, and ease of use. The technologies used are categorized as follows:

◆ 1. Frontend Technologies

- **React.js**
Used to build a dynamic and responsive user interface for the application.
- **Vite**
A fast build tool used for developing and running the frontend efficiently.
- **HTML, CSS, JavaScript**
Used for designing and structuring the web pages.

◆ 2. Backend Technologies

- **Node.js**
A runtime environment used to execute server-side JavaScript.
- **Express.js**
A web framework used to build RESTful APIs and handle server requests.

◆ 3. Authentication & Security

- **JWT (JSON Web Token)**
Used for secure user authentication and session management.
- **Bcrypt**
Used for hashing passwords to enhance security.

◆ 4. Database

- **MongoDB**
A NoSQL database used to store user data, issues, and other system information.
- **JSON Storage (for development)**

- Used during initial development for simple data handling.

◆ 5. Architecture & Communication

- **Microservices Architecture**

The system is divided into multiple independent services for better scalability.

- **REST APIs**

Used for communication between frontend and backend services.

- **API Gateway**

Acts as a central entry point for routing requests to different services.

◆ 6. DevOps & Tools

- **Docker**

Used for containerizing applications and ensuring consistent environments.

- **Docker Compose**

Used to manage multiple containers and services.

- **Git & GitHub**

- Used for version control and project management.

◆ 7. Development Tools

- **Visual Studio Code**

Code editor used for development.

- **Postman**

Used for testing APIs.

METHODOLOGY

The methodology of the Campus Issue Raiser system describes the step-by-step process followed to design, develop, and implement the application. The system follows a structured approach to ensure efficient issue handling and smooth communication between users and administrators.

1. Requirement Analysis

In this phase, the problems in existing systems were identified, such as lack of transparency, no tracking mechanism, and delayed responses. Based on these issues, the requirements for a centralized issue management system were defined.

2. System Design

The system is designed using a microservices architecture. The overall structure includes frontend, API gateway, backend services, and database. Each module is designed to perform a specific function such as user management, issue handling, and interaction.

3. Development

The development phase involves building both frontend and backend components:

- Frontend is developed using React.js for user interaction
- Backend services are built using Node.js and Express.js
- APIs are created for communication between components

4. Implementation of Features

Key features of the system are implemented, including:

- User registration and login
- Issue creation and tracking
- Comment and support system
- Issue status updates
- Escalation of unresolved issues

5. Testing

The system is tested to ensure proper functionality. Different types of testing such as unit testing and API testing are performed using tools like Postman to verify that all modules work correctly.

6. Deployment

The application is deployed using Docker and Docker Compose, which helps in running multiple services efficiently in a containerized environment.

7. Maintenance and Improvement

After deployment, the system is monitored for performance and errors. Necessary updates and improvements are made to enhance functionality and user experience.

Chapter- 7

IMPLEMENTATION

The implementation phase involves transforming the designed architecture of the Campus Issue Raiser system into a fully functional application. The system is developed using a microservices-based approach, where each module is independently implemented and later integrated to form the complete system.

7.1 System Setup

The development environment is set up using Node.js for backend services and React.js for the frontend. Required dependencies and libraries are installed using package managers such as npm. Docker is used to containerize services for consistent deployment.

7.2 Frontend Implementation

The frontend is developed using React.js to provide a responsive and user-friendly interface. The application is structured into reusable components and pages.

Main Modules:

- **Login & Registration Page:** Allows users to create accounts and log in securely
- **Dashboard:** Displays all issues and their current status
- **Issue Submission Page:** Enables users to report issues with details
- **Admin Panel:** Allows administrators to manage and resolve issues

The frontend communicates with backend services using REST APIs and handles user interactions efficiently.

7.3 Backend Implementation

The backend is developed using Node.js and Express.js. It is divided into multiple microservices, each responsible for a specific functionality.

Microservices Implemented:

- **User Service:** Handles user registration, login, and authentication
- **Issue Service:** Manages issue creation, updates, and retrieval
- **Interaction Service:** Handles comments and support (upvotes)
- **Escalation Service:** Automatically escalates unresolved issues
- **Analytics Service:** Generates reports and insights

Each service runs independently and communicates via APIs.

7.4 API Gateway

An API Gateway is implemented to act as a central point for all incoming requests. It routes requests to appropriate services and ensures secure and efficient communication between frontend and backend.

7.5 Database Implementation

MongoDB is used as the primary database to store:

- User information
- Issue details
- Comments and interactions
- Logs and system data

During development, JSON files are used for simple data storage and testing.

7.6 Authentication and Security

Authentication is implemented using JWT (JSON Web Token). When users log in, a token is generated and used for secure communication. Passwords are encrypted using Bcrypt to ensure data protection.

7.7 Functional Implementation

The system includes the following functionalities:

- **User Authentication:** Secure login and registration
- **Issue Management:** Users can create, view, and track issues
- **Interaction System:** Users can comment and support issues
- **Status Management:** Admins update issue status (Pending, In Progress, Resolved)
- **Escalation Mechanism:** Issues are escalated if not resolved on time

7.8 Testing and Debugging

Testing is performed to ensure system reliability:

- API testing using Postman
- Functional testing of frontend components
- Debugging errors and fixing bugs

, ensuring scalability and easy management. The system can be deployed on cloud platforms for real-world usage

Chapter 8

RESULT

Campus Issue Feed

Report clearly. Resolve faster.

FEED

Home

Academics

Hostel

Transport

General

WORKSPACE

Insights

Profile

Admin Panel

Create Issue

Admin Command Center

Every report improves campus life.

Total Issues

0

Resolved

0

Pending

0

Escalated

0

Critical Pressure

0

Neglected > 2 Days

0

Smart Actions

One-click presets for fast operations.

Focus Escalated

Review Resolved

Show All Pending Risk

Auto-select Top Pressure

Backlog Simulator

Estimate clear-time from current throughput.

Issues resolved per day

Throughput: 4/day

Estimated backlog clearance: 0 days

Section Heatmap

Academics

Healthy

Health Index: 100

Open 0

Esc 0

Res 0

P 0

Hostel

Healthy

Health Index: 100

Open 0

Esc 0

Res 0

P 0

Transport

Healthy

Health Index: 100

Open 0

Esc 0

Res 0

P 0

General

Healthy

Health Index: 100

Open 0

Esc 0

Res 0

P 0

All Sections

All Status

System

Raise Issue

Priority Radar

Live Timeline

login · user-service

26/3/2026, 6:09:32 pm

login · user-service

25/3/2026, 8:24:25 pm

Action Logs

Search logs

login

User-service

26/3/2026, 6:09:32 pm

login

User-service

25/3/2026, 8:24:25 pm

Home

Every report improves campus life.

System

Raise Issue

Hi System

Your voice matters. Keep reports clear, calm, and specific for faster action.

Campus Pulse Live

0 active stories · 0 resolved wins

0

Pending

0

Resolved

0

Critical

Home

Academics

Hostel

Transport

General

Search issues, tags, sections

Impact Feed 0

Create Issue

Campus Stories

No stories yet.

Trending

Latest

Pending

Resolved

Section: Home

No issues match current filters.

Trending Tags

No tags yet.

High Pressure

No issues yet.

DEPT OF CSE, SUIET, MUKKA

18

Profile Dashboard

Every report improves campus life.

System   [Raise Issue](#)

System Admin
 admin@campus.local
 admin · General

Issues Raised

0

Resolved Raised

0

Assigned To Me

0

Supports Given

0

Comments Posted

0

Evidence Uploaded

0

My Recent Issues

No issues raised yet.

[Raise New Issue](#)

Create Issue

Every report improves campus life.

Report a campus issue in 3 quick steps

Clear details = faster resolution.

Draft quality

5%

High quality reports with context and evidence are solved faster.

1 Describe the problem

Be specific about what, where, and impact.

Title

Description

Section

Academics

2 Add context

Tags, image, and anonymous mode are optional but helpful.

Where exactly?

When did it start?

How many students affected?

Tags (comma separated)

wifi, attendance

Issue Image (optional)

Add image

Tip: image proof usually increases trust and faster resolution.

☐ Post this issue anonymously for safety

3 Tag responsible authorities

Tagged people receive in-app notifications.

Tag Authorities (optional)

System Admin (admin)

Academic Teacher (teacher)

Hostel Warden (teacher)

Department HOD (hod)

Dean Office (dean)

Campus Management (management)

Ready to publish this as a community-impact issue?

[Submit & Start Resolution](#)

Chapter-9

ADVANTAGES & LIMITATIONS

Advantages

1. Centralized Issue Management

The system provides a single platform where all campus-related issues can be reported and managed efficiently.

2. Transparency and Tracking

Users can track the status of their issues in real time, ensuring visibility and reducing confusion.

3. Improved Communication

It enhances communication between students and authorities through comments and updates.

4. Accountability

Authorities are held accountable as all actions and updates are recorded in the system.

5. Time Efficiency

Issues are handled faster compared to traditional manual methods.

6. Scalable Architecture

The use of microservices allows the system to scale easily and handle increasing users.

7. User-Friendly Interface

The system is designed to be simple and easy to use for all users.

Limitations

1. Dependency on Internet Connectivity

The system requires a stable internet connection to function properly.

2. Initial Setup Complexity

Microservices architecture and API gateway setup can be complex for beginners.

3. Limited Security Features

Basic authentication is implemented; advanced security mechanisms can be improved.

4. Shared Database Approach

Using a single database for multiple services reduces true microservices independence.

5. No Real-Time Notifications

The system currently does not support instant notifications for updates.

6. Limited Error Handling

Some edge cases and failures may not be fully handled in the current version.

Chapter-10

FUTURE ENHANCEMENT

The Campus Issue Raiser system can be further improved by adding advanced features and technologies to enhance performance, usability, and scalability. Some possible future enhancements are:

1. **Real-Time Notifications**

Implement real-time alerts using technologies like WebSockets to notify users instantly about issue updates, comments, and status changes.

2. **Mobile Application Development**

Develop a mobile app for Android and iOS platforms to make the system more accessible and user-friendly.

3. **Advanced Security Mechanisms**

Enhance security by implementing OAuth authentication, multi-factor authentication (MFA), and better data encryption techniques.

4. **Event-Driven Architecture**

Introduce message brokers like Kafka or RabbitMQ to enable asynchronous communication between microservices.

5. **AI-Based Issue Prioritization**

Use artificial intelligence to automatically categorize and prioritize issues based on urgency and impact.

6. **Separate Databases for Microservices**

Assign individual databases to each microservice to achieve complete microservices independence.

7. **Performance Monitoring and Logging**

Integrate tools for monitoring system performance and logging errors to improve reliability and maintenance.

Chapter-11

CONCLUSION

The Campus Issue Raiser system provides an effective and modern solution for managing campus-related issues in a structured and transparent manner. It overcomes the limitations of traditional complaint systems by offering a centralized platform where users can easily report, track, and interact on issues.

The system successfully implements key features such as issue tracking, user interaction, status updates, and escalation mechanisms. These features improve communication between students and authorities and ensure better accountability in resolving problems.

By using a microservices-based architecture and modern web technologies, the system achieves scalability, flexibility, and ease of maintenance. It also provides a user-friendly interface, making it accessible and efficient for all users.

Although the current system has some limitations, it has strong potential for further enhancement and real-world implementation. With the addition of advanced features like real-time notifications and improved security, the system can become a complete solution for campus issue management.

In conclusion, the Campus Issue Raiser system plays an important role in improving transparency, efficiency, and responsiveness in handling campus issues, thereby contributing to a better and more organized institutional environment.

Chapter-12

REFERENCES

- Google, *Gemini API Documentation*. Available: <https://ai.google.dev/>
- Node.js Official Documentation. Available: <https://nodejs.org/>
- Express.js Documentation. Available: <https://expressjs.com/>
- React Official Documentation. Available: <https://react.dev/>
- MongoDB Documentation. Available: <https://www.mongodb.com/docs/>
- GitHub Guides. Available: https://github.com/pdprajwal9522-cloud/ibm_project.git
- , Research Articles and Online Resources.